Formal Languages — Final Project

FIRST and FOLLOW sets, $LL(1)$ and $SLR(1)$ parsers,
and a syntax-directed definition

Juan Manuel Zuluaga Santiago Arboleda Giraldo Nicolás Patiño Palacio

November 11, 2026

Abstract

This report describes our implementation of the final project. We implemented, in Python 3 and without any parser generator or parsing library, the computation of the FIRST and FOLLOW sets of a context-free grammar, the table-driven $LL(1)$ parser, the $SLR(1)$ parser built from the canonical collection of $LR(0)$ item sets, and a generic evaluator of syntax-directed definitions that works with the dependency graph and a topological order. Every algorithm follows Aho et al., *Compilers: Principles, Techniques, and Tools* (2nd edition); the section, algorithm or figure that each part follows is stated below and is repeated in the docstring of the corresponding module.

Contents

1 What of the book we followed, and with which data structures **1**
- 1.1 Sections and algorithms . . . 1
- 1.2 Data structures . . . 1

2 Items 1 and 2 **3**
- 2.1 The grammar we used . . . 3
- 2.2 FIRST and FOLLOW (Item 1) . . . 3
- 2.3 Predictive parsing table (Algorithm 4.31) . . . 3
- 2.4 Canonical collection of $LR(0)$ item sets (Section 4.6.2) . . . 3
- 2.5 ACTION and GOTO tables (Algorithm 4.46) . . . 4
- 2.6 A string parsed step by step . . . 5

3 Item 3: the syntax-directed definition **5**
- 3.1 Step 1: the grammar . . . 5
- 3.2 Step 2.1: the attributes . . . 6
- 3.3 Step 2.2: the semantic rules . . . 6
- 3.4 Step 3: an example by hand . . . 7
- 3.5 Step 4: classification of the SDD . . . 8
- 3.6 Step 5: the implementation . . . 9
- 3.7 Step 6: tests . . . 10

4 Extra: web interface **10**

5 Problems we found, and how we solved them **11**

6 How to run it 12

1 What of the book we followed, and with which data structures

1.1 Sections and algorithms

Table 1 lists, part by part, what we read and implemented.

Part	File	Section / algorithm of the book
FIRST and FOLLOW	`sets.py`	Section 4.4.2. The three rules for FIRST(X), the extension of FIRST to a string $X_1X_2\cdots X_n$, and the three rules for FOLLOW(A). Both are computed by iteration until nothing changes.
$LL(1)$ condition	`ll1.py`	The condition stated at the beginning of Section 4.4.3, tested on every pair of alternatives of every nonterminal.
Predictive parsing table	`ll1.py`	Algorithm 4.31 (Section 4.4.3), Figure 4.17 for the shape of the table.
$LL(1)$ parser	`ll1.py`	Algorithm 4.34, the table-driven predictive parser of Section 4.4.4 (Figure 4.19), with the trace in the format of Figure 4.21.
$LR(0)$ items	`slr1.py`	Section 4.6.2: CLOSURE(I) and GOTO(I, X) of Figure 4.32, and the canonical collection of Figure 4.33, on the augmented grammar $S' \to S$.
$SLR(1)$ tables	`slr1.py`	Algorithm 4.46 (Section 4.6.4), which uses FOLLOW. The grammar is $SLR(1)$ iff no cell of ACTION receives two different entries (Section 4.5.4 for the kinds of conflict).
$SLR(1)$ parser	`slr1.py`	Algorithm 4.44, the LR parsing algorithm of Section 4.6.3 (Figure 4.36), with the trace in the format of Figure 4.38.
SDD	`sdd.py`	Section 5.1.1 (synthesized and inherited attributes) and the format of Figure 5.1 for the semantic rules.
Evaluation	`evaluator.py`	Section 5.2.1 (dependency graph) and Section 5.2.2 (evaluation order as a topological sort; a cycle means the SDD is circular). The classification uses Sections 5.2.3 and 5.2.4.

Table 1: Parts of the project and the place in the book that each one follows.

1.2 Data structures

- **Grammar.** A list of pairs `(head, body)`, where the body is a tuple of one-character symbols and the empty tuple `()` represents ε. The position in that list is the *number* of the production and is what the two tables store, so a table entry costs one integer. A dictionary `head` $\to$ `list of indices` gives the alternatives of a nonterminal without scanning the list.

- **First and Follow.** Dictionaries `symbol` → `set of terminals`. ε is stored inside the set as the string `e` and the end marker as `$`, exactly as in the notation of the assignment, so printing them is immediate. Using sets makes the fixed-point loop easy to write: we repeat the rules until no set changes size.
- $LL(1)$ **table.** A dictionary `(A, a)` → `list of production numbers`. The value is a *list* and not a single number on purpose: a cell with more than one element is a conflict, which is how the table detects that the grammar is not $LL(1)$.
- $LR(0)$ **items and states.** An item is the pair `(production number, position of the dot)`. A state is the `frozenset` of its items, so two states with the same items are automatically the same state and can be used as dictionary keys. The canonical collection is a list of those frozensets and Goto is a dictionary `(state, symbol)` → `state`.
- **Action table.** A dictionary `(state, terminal)` → `(kind, value)` with kind in `{shift, reduce, accept}`. Writing an entry over a different one is what we record as a conflict.
- **Parse tree.** A node stores its symbol, the number of the production used (only for nonterminals), its children and a dictionary of attribute values. Both parsers build the same kind of tree, so Item 3 works with either of them.
- **Dependency graph.** An attribute instance is the pair `(number of the node, name of the attribute)`; the nodes are numbered in preorder before building the graph so that two nodes with the same symbol are never confused. The graph itself is a set of edges, and the topological sort is Kahn's algorithm over a counter of incoming edges.

2 Items 1 and 2

2.1 The grammar we used

The grammar below is ours; it is not one of the examples of the assignment. It describes a block that contains a list of items, where an item is either an identifier `i` or another block. It is recursive, it has an ε-production, and it is both $LL(1)$ and $SLR(1)$, so both parsers can be shown on it.

```
3
S -> {L}
L -> IL e
I -> {L} i
```

In the notation of the book:

$$S \to \texttt{\{}\, L \,\texttt{\}} \qquad L \to I\,L \mid \varepsilon \qquad I \to \texttt{\{}\, L \,\texttt{\}} \mid \texttt{i}$$

2.2 First and Follow (Item 1)

Output of `python main.py --sets` on that grammar:

```
First(I) = {i, {}
First(L) = {i, {, e}
First(S) = {{}
Follow(I) = {i, {, }}
Follow(L) = {}}
Follow(S) = {$}
```

Note how ε is handled: L has the production $L \to \varepsilon$, so `e` belongs to $\textsc{First}(L)$. Because L is nullable, in $L \to IL$ the symbol I is followed by $\textsc{First}(L) \setminus \{\varepsilon\} = \{\texttt{i}, \texttt{\{}\}$ (rule 2) and, since L can disappear, also by everything in $\textsc{Follow}(L)$ (rule 3). That is why $\textsc{Follow}(I) = \{\texttt{i}, \texttt{\{}, \texttt{\}}\}$. In turn, $\textsc{Follow}(L) = \{\texttt{\}}\}$ comes from rule 2 applied to $S \to \texttt{\{}L\texttt{\}}$ and to $I \to \texttt{\{}L\texttt{\}}$, where L is always followed by `}`. The end marker `$` appears only in $\textsc{Follow}(S)$, by rule 1.

2.3 Predictive parsing table (Algorithm 4.31)

```
        i          {          }          $
I     I -> i    I -> {L}
L    L -> IL    L -> IL     L -> e
S               S -> {L}
```

No cell holds two productions, and the pairwise test of Section 4.4.3 passes as well: for I, $\textsc{First}(\{L\}) = \{\texttt{\{}\}$ and $\textsc{First}(\texttt{i}) = \{\texttt{i}\}$ are disjoint; for L, $\textsc{First}(IL) = \{\texttt{i}, \texttt{\{}\}$ does not meet $\textsc{Follow}(L) = \{\texttt{\}}\}$, which is the condition that the ε alternative has to satisfy. The grammar is therefore $LL(1)$.

2.4 Canonical collection of $LR(0)$ item sets (Section 4.6.2)

The grammar is augmented with $S' \to S$. The collection our program computes has eleven states:

```
I0:
    S' -> .S
    S -> .{L}
    GOTO(I0, S) = I1
    GOTO(I0, {) = I2

I1:
    S' -> S.

I2:
    S -> {.L}
    L -> .IL
    L -> .
    I -> .{L}
    I -> .i
    GOTO(I2, I) = I3
    GOTO(I2, L) = I4
    GOTO(I2, i) = I5
    GOTO(I2, {) = I6

I3:
    L -> .IL
    L -> I.L
    L -> .
    I -> .{L}
    I -> .i
    GOTO(I3, I) = I3
    GOTO(I3, L) = I7
    GOTO(I3, i) = I5
    GOTO(I3, {) = I6

I4:
    S -> {L.}
    GOTO(I4, }) = I8

I5:
    I -> i.

I6:
    L -> .IL
    L -> .
    I -> .{L}
    I -> {.L}
    I -> .i
    GOTO(I6, I) = I3
    GOTO(I6, L) = I9
    GOTO(I6, i) = I5
    GOTO(I6, {) = I6

I7:
    L -> IL.

I8:
    S -> {L}.

I9:
    I -> {L.}
    GOTO(I9, }) = I10

I10:
    I -> {L}.
```

2.5 Action and Goto tables (Algorithm 4.46)

```
Numbered productions:
    1. S -> {L}
    2. L -> IL
    3. L -> e
```

```
    4. I -> {L}
    5. I -> i
```

STATE	i	{	}	$	I	L	S
0		s2					1
1				acc			
2	s5	s6	r3		3	4	
3	s5	s6	r3		3	7	
4			s8				
5	r5	r5	r5				
6	s5	s6	r3		3	9	
7			r2				
8				r1			
9			s10				
10	r4	r4	r4				

States 2, 3 and 6 are the interesting ones: they contain the complete item $L \to \cdot$ and also items with the dot before a terminal. There is no shift/reduce conflict because the reduction $L \to \varepsilon$ is written only in the columns of $\textsc{Follow}(L) = \{\texttt{}\}$, while the shifts are on `i` and `{`. This is exactly the difference between an $LR(0)$ and an $SLR(1)$ table. No cell has two entries, so the grammar is $SLR(1)$.

2.6 A string parsed step by step

We parse `{i{i}}` with both parsers. With the $LL(1)$ parser (format of Figure 4.21; the stack is written bottom-to-top):

STACK	INPUT	ACTION
$S	{i{i}}$	output S -> {L}
$}L{	{i{i}}$	match {
$}L	i{i}}$	output L -> IL
$}LI	i{i}}$	output I -> i
$}Li	i{i}}$	match i
$}L	{i}}$	output L -> IL
$}LI	{i}}$	output I -> {L}
$}L}L{	{i}}$	match {
$}L}L	i}}$	output L -> IL
$}L}LI	i}}$	output I -> i
$}L}Li	i}}$	match i
$}L}L	}}$	output L -> e
$}L}	}}$	match }
$}L	}$	output L -> e
$}	}$	match }
$	$	accept

With the $SLR(1)$ parser (format of Figure 4.38):

STACK	SYMBOLS	INPUT	ACTION
0		{i{i}}$	shift 2
0 2	{	i{i}}$	shift 5
0 2 5	{i	{i}}$	reduce I -> i
0 2 3	{I	{i}}$	shift 6
0 2 3 6	{I{	i}}$	shift 5

```
0 2 3 6 5            {I{i          }}$          reduce I -> i
0 2 3 6 3            {I{I          }}$          reduce L -> e
0 2 3 6 3 7          {I{IL         }}$          reduce L -> IL
0 2 3 6 9            {I{L          }}$          shift 10
0 2 3 6 9 10         {I{L}         }$           reduce I -> {L}
0 2 3 3              {II           }$           reduce L -> e
0 2 3 3 7            {IIL          }$           reduce L -> IL
0 2 3 7              {IL           }$           reduce L -> IL
0 2 4                {L            }$           shift 8
0 2 4 8              {L}           $            reduce S -> {L}
0 1                  S             $            accept
```

The two parsers accept the same strings, which is what one expects: the top-down parser produces the leftmost derivation from the root down, and the bottom-up parser produces the reverse of a rightmost derivation.

3 Item 3: the syntax-directed definition

3.1 Step 1: the grammar

```
5
S -> N#B
N -> DR
R -> DR e
D -> 0 1 2
B -> b t
```

A string of this language is a sequence of digits, then `#`, then one letter that says in which base the digits must be read: `b` for base 2 and `t` for base 3. For instance `102#t` is the numeral 102 in base 3, which is 11, and `1011#b` is 1011 in base 2, which is also 11.

The grammar satisfies the three conditions of Step 1:

G1 It follows the conventions of Section 2.2 and our own program reports that it is *both* $LL(1)$ and $SLR(1)$, so the strings are parsed with the parsers of Item 2. (The evaluator accepts either one; the tests check that both give the same values.)

G2 It has five nonterminals and nine productions, and R is recursive.

G3 Neither the grammar nor the SDD is in the book or in the assignment. The idea of accumulating a value from left to right is taken from the book, but the computation is different: here the *base* is not fixed, it is read at the end of the string and travels back to the first digit.

3.2 Step 2.1: the attributes

There are three different attributes, *val* (synthesized), *base* and *acc* (both inherited), and the start symbol S has no inherited attribute, as the assignment requires.

3.3 Step 2.2: the semantic rules

The two checks of Section 5.1.1 hold for every production:

- Every rule defines either a synthesized attribute of the head or an inherited attribute of a symbol of the body, and uses only attributes of symbols of that same production.

Symbol	Attribute	Kind	Type	Meaning
S	*val*	synthesized	integer	value of the whole numeral
N	*base*	inherited	integer	base in which the digits must be read
N	*acc*	inherited	integer	value accumulated before the first digit
N	*val*	synthesized	integer	value of the whole list of digits
R	*base*	inherited	integer	base in which the digits must be read
R	*acc*	inherited	integer	value of the digits already read
R	*val*	synthesized	integer	accumulated value when the list ends
D	*val*	synthesized	integer	value of the digit
B	*val*	synthesized	integer	base named by the final letter
terminal	*lexval*	synthesized	character	the character itself

Table 2: Attributes of the SDD. Terminals only have the synthesized attribute *lexval*, which the parser already knows, so no scanner is needed.

Production	Semantic rules
$S \to N\,\texttt{\#}\,B$	$N.base = B.val$; $N.acc = 0$; $S.val = N.val$
$N \to D\,R$	$R.base = N.base$; $R.acc = N.acc \times N.base + D.val$; $N.val = R.val$
$R \to D\,R_1$	$R_1.base = R.base$; $R_1.acc = R.acc \times R.base + D.val$; $R.val = R_1.val$
$R \to \varepsilon$	$R.val = R.acc$
$D \to \texttt{0}$	$D.val = digit(\texttt{0}.lexval)$
$D \to \texttt{1}$	$D.val = digit(\texttt{1}.lexval)$
$D \to \texttt{2}$	$D.val = digit(\texttt{2}.lexval)$
$B \to \texttt{b}$	$B.val = base(\texttt{b}.lexval) = 2$
$B \to \texttt{t}$	$B.val = base(\texttt{t}.lexval) = 3$

Table 3: The SDD, in the format of Figure 5.1 of the book.

- In every production, each synthesized attribute of the head and each inherited attribute of each nonterminal of the body is defined exactly once. In $S \to N\#B$, for example, $S.val$ is defined once and the two inherited attributes of N are defined once each; B has no inherited attributes, so there is nothing to define for it.

3.4 Step 3: an example by hand

We take the string `102#t`, which has five terminals. Its parse tree, annotated with the value of every attribute, is shown in Figure 1. The number in brackets is the number our program gives to the node (preorder), and it is used to name the attribute instances in the dependency graph.

The dependency graph (Section 5.2.1) is shown in Figure 2. There is one vertex per attribute of each node and one edge from every attribute that a rule uses to the attribute that the rule defines. The vertices are drawn by levels, so that every edge goes downwards.

A valid evaluation order is any topological order of that graph. The one our program produces is the following (it is the order printed by `sdd_main.py`):

```
 1. N[1].acc     = 0          12. R[4].acc    = 1
 2. 1[3].lexval  = '1'        13. R[4].base   = 3
```

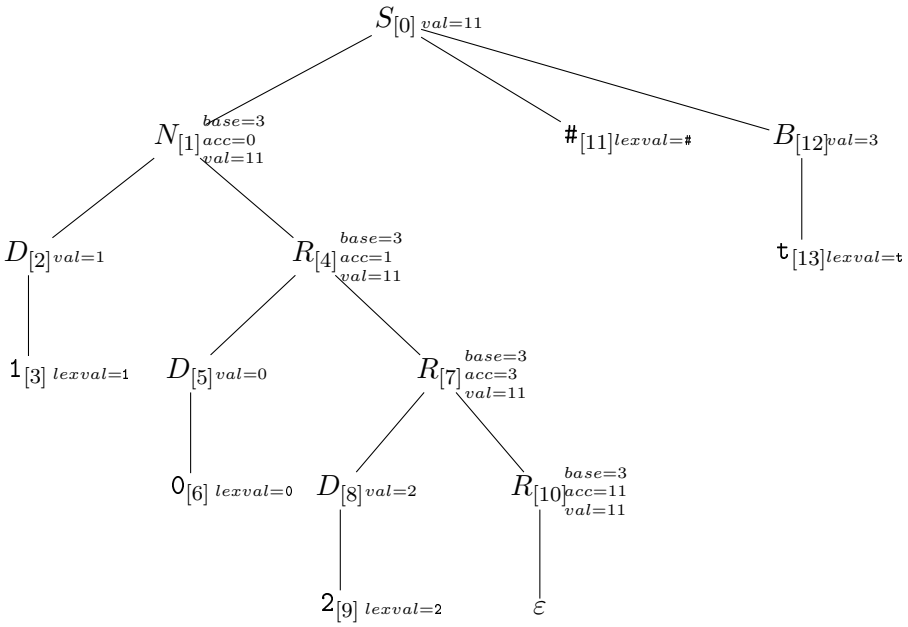


Figure 1: Annotated parse tree of `102#t`. The value of the main attribute is $S.val = 11$, because $1 \cdot 3^2 + 0 \cdot 3 + 2 = 11$.

```
 3. D[2].val     = 1          14. R[7].acc    = 3
 4. 0[6].lexval  = '0'        15. R[7].base   = 3
 5. D[5].val     = 0          16. R[10].acc   = 11
 6. 2[9].lexval  = '2'        17. R[10].base  = 3
 7. D[8].val     = 2          18. R[10].val   = 11
 8. #[11].lexval = '#'        19. R[7].val    = 11
 9. t[13].lexval = 't'        20. R[4].val    = 11
10. B[12].val    = 3          21. N[1].val    = 11
11. N[1].base    = 3          22. S[0].val    = 11
```

The important step is number 11: $N_{[1]}.base$ can only be computed after $B_{[12]}.val$, which belongs to a node that is *to the right* of N in the tree. Everything that accumulates the digits has to wait for it.

3.5 Step 4: classification of the SDD

Our SDD is **neither S-attributed nor L-attributed**.

- It is not S-attributed (Section 5.2.3): an S-attributed SDD has only synthesized attributes, and ours has the inherited attributes $N.base$, $N.acc$, $R.base$ and $R.acc$.
- It is not L-attributed (Section 5.2.4). The definition requires that, in a production $A \to X_1 X_2 \cdots X_n$, every inherited attribute of X_i depend only on inherited attributes of A, on attributes of $X_1, \ldots, X_{i-1}$, or on attributes of X_i itself in a non-circular way. In the production

$$S \to \underbrace{N}_{X_1}\ \underbrace{\#}_{X_2}\ \underbrace{B}_{X_3}, \qquad N.base = B.val,$$

  the inherited attribute of X_1 depends on an attribute of X_3, which is to its right. That is exactly what an L-attributed definition forbids. All the other productions do satisfy the condition, but one counterexample is enough.

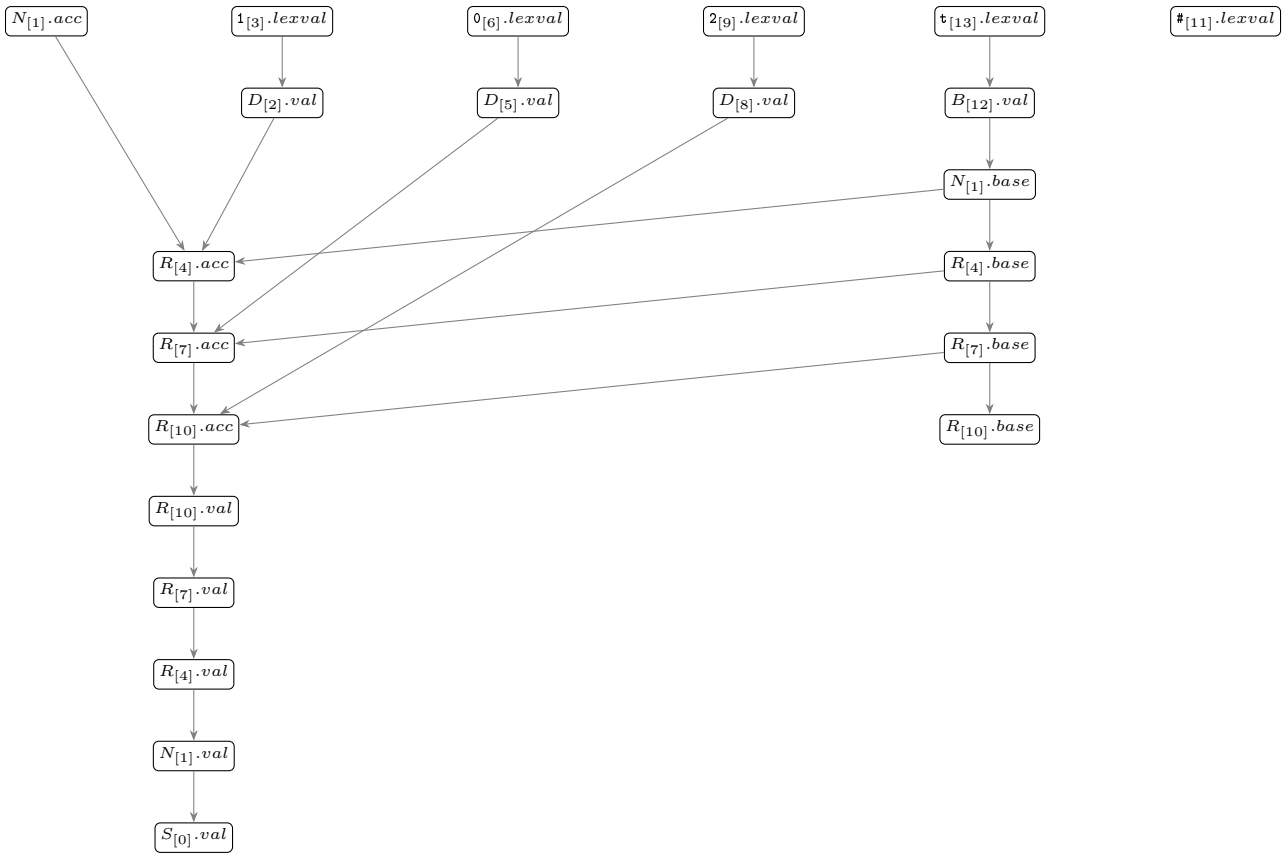


Figure 2: Dependency graph of `102#t`. The vertex $\#_{[11]}.lexval$ is isolated: the semantic rules never use it. The vertex $R_{[10]}.base$ is defined but never used either, because $R_{[10]}$ applies $R \to \varepsilon$.

This was deliberate. An L-attributed SDD can be evaluated with a single recursive pass over the tree that visits the children from left to right, so the general method would never be exercised. Here that pass fails — when it reaches N it does not yet know the base — while the dependency graph and the topological order work without any change, which is the point of the method of Section 5.2.2. The SDD is still non-circular, so a valid evaluation order always exists.

3.6 Step 5: the implementation

The implementation follows the five points of Step 5 of the assignment.

1. **The SDD is data, not code.** The file `sdd.py` holds a dictionary that maps every production to its list of semantic rules, and each rule stores the attribute it defines, the list of attributes it uses, and the function that computes it. Attributes are referred to by *position* inside the production: 0 is the head, 1 is the first symbol of the body, and so on. The rule $R.acc = N.acc \times N.base + D.val$, for instance, is stored as the target $(2, acc)$, the dependencies $[(0, acc), (0, base), (1, val)]$ and the function `lambda v: v[0]*v[1] + v[2]`.

2. **Parse tree.** The string is parsed with the $LL(1)$ parser of Item 2 (or with the $SLR(1)$ one, with the flag `--slr`), and the parser builds the tree as it works: the top-down parser creates the children when it expands a nonterminal, and the bottom-up parser pops $|\beta|$ nodes and makes them the children of the new node when it reduces by $A \to \beta$.

3. **Dependency graph.** `evaluator.py` walks the tree, and for every internal node it looks up the rules of its production and instantiates them: the positions are replaced by the actual nodes, and for every dependency an edge is added from the attribute that is used to the attribute that is defined. Leaves contribute their *lexval*, which the parser already filled in.

4. **Order and evaluation.** Kahn's algorithm gives a topological order; among the vertices that are ready we always take the smallest one, so that the output is the same in every run. If some vertex never becomes ready, the graph has a cycle, and the program reports that the SDD is circular and stops without evaluating anything. Otherwise the attributes are computed in that order.

5. **Output.** For every string the program prints `no`, or the annotated tree (one node per line, indented by depth), the evaluation order and the value of the main attribute $S.val$.

`evaluator.py` never mentions *base*, *acc* or *val*: replacing the dictionary of `sdd.py` is enough to evaluate a completely different SDD, which is what the assignment asks for.

3.7 Step 6: tests

Table 4 shows the strings we used. They include the shortest string of the language, a string that is not in the language, and a string with deep recursion. All of them are also in the automatic test suite (`python tests.py`).

String	Result	Why
`0#b`	$S.val = 0$	shortest string of the language (one digit)
`102#t`	$S.val = 11$	the example worked out by hand
`1011#b`	$S.val = 11$	the same value written in base 2
`210201#t`	$S.val = 586$	six digits, so the recursion on R is six levels deep
`102`	`no`	the base is missing
`121#x`	`no`	`x` is not a terminal of the grammar
`e` (empty)	`no`	the language has no empty string

Table 4: Test strings and their results.

Circular SDD. To show that the evaluator detects circularity, we changed one rule of the production $R \to D\,R_1$: instead of $R_1.acc = R.acc \times R.base + D.val$ we wrote $R_1.acc = R.val$. Since $R.val = R_1.val$ and $R_1.val$ ends up depending on $R_1.acc$, the graph gets the cycle

$$R.val \;\to\; R_1.acc \;\to\; R_1.val \;\to\; R.val.$$

That version is kept in `sdd.py` as `CIRCULAR_SDD` and is used with the flag `--circular`. Running `python sdd_main.py --circular` on `102#t` prints:

```
Cadena: 102#t
La SDD es circular: el grafo de dependencias tiene un ciclo.
Atributos que quedaron sin poder evaluarse:
    S[0].val
    N[1].val
    R[4].val
    R[7].acc
```

```
R[7].val
R[10].acc
R[10].val
```

On the string `0#b` the same circular SDD evaluates normally, because that tree never uses the production $R \to D\,R_1$ and therefore never builds the cycle. Circularity is a property of each particular tree, not of the SDD alone, which matches what Section 5.2.2 says.

4 Extra: web interface

Besides the two console programs, we built a small web application. It does not implement any algorithm of its own: it calls exactly the same functions, so whatever the page shows is what `main.py` and `sdd_main.py` print. We kept the layered architecture we had used in the previous assignment of the course:

Layer	File	Responsibility
Controller	`app.py`	Flask routes. Validates the shape of the request and turns results or errors into HTTP responses.
Gateway	`gateway.py`	Turns the internal structures (sets, frozensets of items, tree nodes) into JSON, and every domain error into a `GatewayError`.
Algorithms	Items 1–3	Unchanged. They know nothing about HTTP or Flask.
View	`analyzer.html`	One page, no external dependencies.

The endpoints are `POST /api/grammar` (sets, tables, $LR(0)$ automaton and verdict), `POST /api/parse` (parses strings and returns the trace and the tree), `POST /api/sdd` (evaluates the SDD) and `GET /analyzer` (the page). A malformed request gets a `400` and one that the algorithms reject gets a `422`, always as JSON.

The page is worth it for two things that are hard to read in a terminal. The parse tree is drawn as SVG with every node annotated with its attributes, which is the annotated tree of Figure 1 but generated automatically for any string. And the dependency graph is drawn by levels, with one level per step of the topological order, so that every edge goes downwards and the shape of the graph is visible at a glance; when the circular SDD is selected, the attributes that end up inside the cycle are drawn in red. Making the drawing forced us to be precise about the direction of the edges, because a graph drawn with the edges reversed looks obviously wrong on screen.

5 Problems we found, and how we solved them

1. **First of a string versus First of a symbol.** Our first version only computed $\textsc{First}(X)$ for single symbols, and the $LL(1)$ table came out wrong for productions whose body starts with a nullable nonterminal. The book extends First to strings $X_1 \cdots X_n$ in Section 4.4.2 and that extension is what Algorithm 4.31 really needs. We wrote it as a separate function (`first_of_string`) and the table became correct.

2. **Where ε belongs and where it does not.** ε can be in a FIRST set but never in a FOLLOW set. We were adding FIRST(β) whole to FOLLOW(B) in rule 2 instead of FIRST(β)$\setminus\{\varepsilon\}$, which produced an `e` inside FOLLOW and, through Algorithm 4.31, a column that does not exist in the table. Writing the two rules with the explicit subtraction fixed it.

3. **The temptation to fix the grammar.** The assignment forbids transforming the grammar before testing whether it is $LL(1)$. With a left-recursive grammar such as $S \to S{+}T \mid T$, the natural reaction is to remove the left recursion so that the $LL(1)$ parser works. We did not: our test simply reports a conflict, which is the honest answer, and that grammar is handled by the $SLR(1)$ parser (it is Example 1 of the assignment).

4. **The offset of the augmented grammar.** In `slr1.py` production number 0 is $S' \to S$, so every production of the original grammar is shifted by one. We first used those numbers directly when building the parse tree and the nodes ended up pointing at the wrong production, which made the SDD evaluator complain about rules that did not exist. The fix is one line (`production=value - 1`), but finding it took a while because the parser kept answering `yes` correctly; only the tree was wrong.

5. **Reproducible output.** Both the canonical collection and the topological order depend on the order in which we iterate over sets, and Python does not promise a fixed order for sets. Two runs could number the states differently, which is confusing when comparing with the book. We iterate over `grammar.symbols()` (a sorted list) when building the collection, and we use a heap in the topological sort so that the smallest ready vertex is always taken first. Now the output is always the same.

6. **Which way the edges go.** The assignment warns about this and we still got it wrong the first time: we drew the edge from the attribute that is *defined* to the attribute that is *used*. The result was a graph that was still acyclic, so nothing crashed — it simply evaluated the attributes in the reverse order and every value came out as `None`. The rule is the one of Section 5.2.1: the edge goes from the attribute that is used to the attribute that is defined.

7. **Choosing an SDD that is worth the general method.** Our first SDD was L-attributed, and we realised that a plain recursive traversal evaluated it correctly, so the dependency graph was not really doing any work. We changed the grammar so that the base appears at the end of the string; now the SDD is neither S-attributed nor L-attributed and the topological order is indispensable.

8. **ε-productions in the tree.** A node for $A \to \varepsilon$ has no children, which breaks any code that assumes that an internal node has at least one child, and in the $SLR(1)$ parser a reduction by an ε-production pops zero states. We decided to keep the list of children empty so that the positions of the semantic rules always match the body of the production, and to print a line `e` only when showing the tree.

6 How to run it

```
python main.py --sets        # Items 1 and 2 (--sets prints FIRST and FOLLOW)
python main.py --tables      # extra: LL(1) table, LR(0) automaton, SLR(1) tables
python main.py --trace       # extra: step-by-step parse of every string
python sdd_main.py           # Item 3
python sdd_main.py --circular    # Item 3 with the circular SDD
```

```
python tests.py              # the test suite (67 checks)
python app.py                # extra: web server, page at /analyzer
```

The `README.md` file explains the input format and the flags in detail.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd edition. Addison-Wesley, 2006. ISBN 0321486811.